

Projektowanie i implementacja skalowalnego systemu AI klasy enterprise wykorzystującego RAG, SQL Agents oraz modele LLM

Celem projektu jest zaprojektowanie i implementacja nowoczesnego systemu AI klasy enterprise, który wykorzystuje modele językowe (LLM), Retrieval-Augmented Generation (RAG), wyszukiwanie semantyczne, vector databases, integrację SQL oraz komunikację przez API.

System ma umożliwiać:

- analizę dokumentów,
- wyszukiwanie wiedzy,
- generowanie odpowiedzi,
- wykonywanie zapytań SQL,
- integrację z systemami firmy,
- oraz automatyzację procesów biznesowych.

Składowe rozwiązania

1. RAG (Retrieval-Augmented Generation) to architektura, w której model LLM nie odpowiada wyłącznie na podstawie wiedzy zapisanej podczas treningu, ale najpierw:

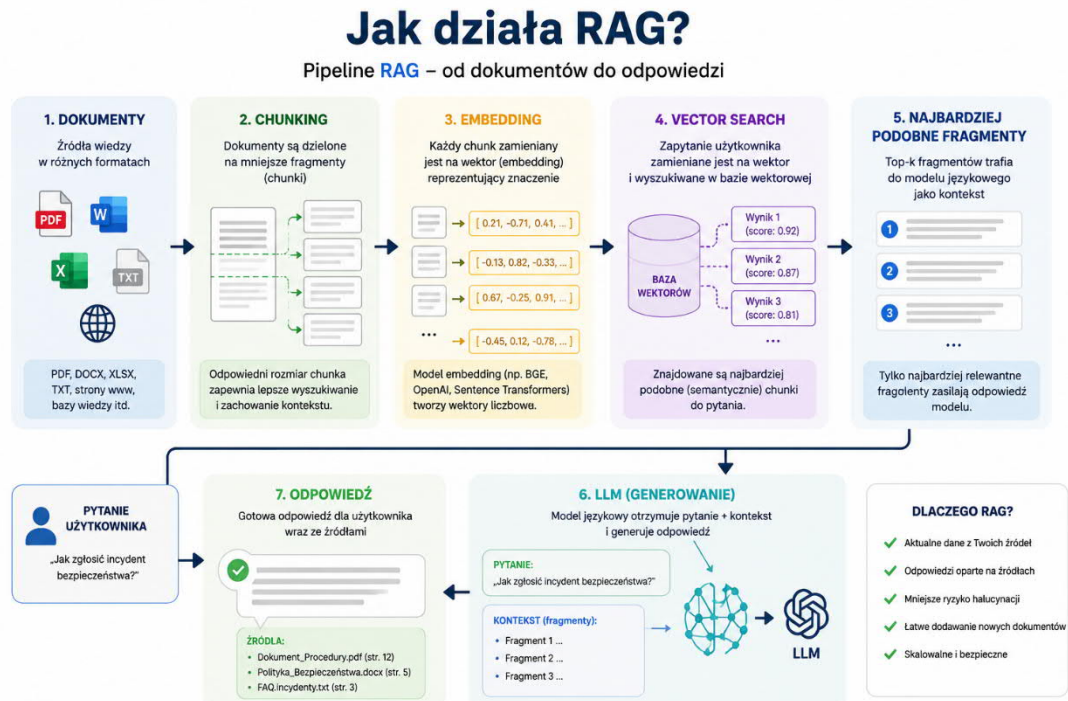
wyszukuje odpowiednie informacje w dokumentach firmy.

Dzięki temu system może:

- analizować PDF,
- procedury,
- regulacje,
- instrukcje,
- dokumentację techniczną,
- umowy,

- polityki bezpieczeństwa.

Jak działa RAG?



Główne zalety RAG

Aktualność danych

LLM nie musi znać dokumentów podczas treningu.

Odpowiedzi oparte o źródła

System może zwrócić:

- odpowiedź,
- cytat,
- źródło dokumentu,
- numer strony.

2. Chunking

Chunking oznacza:

- dzielenie dokumentów na mniejsze fragmenty.

LLM nie analizuje całego PDF jednocześnie.

Dokument jest dzielony np. na:

- akapity,
- sekcje,
- strony,
- okna tekstowe.

Dlaczego chunking jest ważny?

Bo:

- embedding działa na fragmentach,
- retrieval działa na fragmentach,
- zbyt duże chunki pogarszają wyszukiwanie,
- zbyt małe tracą kontekst.

Typowe strategie

Fixed-size chunking

Np:

- 500 tokenów,
- overlap 100.

Semantic chunking

Podział:

- po nagłówkach,
- po sekcjach,
- po znaczeniu.

Sliding window

Nakładające się fragmenty.

3. Embedding

Embedding to:

- wektorowa reprezentacja znaczenia tekstu.

LLM nie wyszukuje słów:

- tylko znaczenie semantyczne.

Przykład

Pytanie:

Jak zgłosić incydent bezpieczeństwa?

System znajdzie także:

- „procedura zgłoszenia naruszenia”,
- „security incident workflow”,
- „cybersecurity escalation”.

Embedding zamienia tekst na liczby

Np:

[0.231, -0.821, 0.554, ...]

Potem można:

- porównywać podobieństwo,
- wyszukiwać semantycznie,
- grupować dokumenty.

4. Vector Search

Vector Search oznacza:

wyszukiwanie podobieństwa embeddingów

Zamiast:

WHERE text LIKE '%incident%'

system używa:

- cosine similarity,
- ANN search,
- nearest neighbors.

Dlaczego to ważne?

Bo użytkownik może zadać pytanie:

- innymi słowami,
- biznesowo,
- nie znając terminologii.

A system i tak znajdzie właściwe dokumenty.

5. FAISS / Vector DB

FAISS to biblioteka Meta AI do:

- przechowywania embeddingów,
- szybkiego wyszukiwania,
- Approximate Nearest Neighbor Search.

Co robi FAISS?

Przechowuje:

miliony embeddingów

i bardzo szybko znajduje:

- najbardziej podobne dokumenty,
- najbliższe fragmenty tekstu.

Dlaczego ANN?

Brute-force search byłby za wolny.

ANN:

- HNSW,
- IVE,
- PQ,

pozwała działać:

- szybko,
- skalowalnie,
- na milionach dokumentów.

6. LLM

LLM generuje odpowiedź na podstawie retrieved context

Czyli:

- dostaje pytanie,
- dostaje znalezione fragmenty,
- generuje odpowiedź.

Kluczowa różnica

Bez RAG:

LLM zgaduje

z RAG:

LLM odpowiada na podstawie dokumentów

AI Agent SQL

AI Agent SQL zamienia:

język naturalny

na:

zapytania SQL

Przykład

Pytanie:

Który region miał najwyższą sprzedaż?

```
SELECT region, SUM(sales)
FROM orders
GROUP BY region
ORDER BY SUM(sales) DESC
```

Co robi agent?

Nie tylko generuje SQL.

Także:

- analizuje wynik,
- interpretuje dane,
- tworzy odpowiedź biznesową,
- może generować wykresy.

8. Pipeline produkcyjny

To:

pełny przepływ danych

od:

- dokumentu,
- do odpowiedzi użytkownika.

Typowy pipeline RAG

Od dokumentu do odpowiedzi użytkownika



Dokumenty
(źródła wiedzy)



AI rozumie
znaczenie



Dokładne odpowiedzi
z kontekstem



Lepsze decyzje
i efektywność

9. Orchestration Layer

To warstwa sterująca całym systemem.

Koordynuje:

- retrieval,
- SQL,
- API,
- memory,
- agents,
- LLM.

Przykład

User:

Ile reklamacji dotyczy procedury X?

System:

- przeszukuje dokumenty,
- odpytuje SQL,
- wywołuje API CRM,
- łączy dane,
- generuje odpowiedź.

10. Semantic Kernel

Semantic Kernel to framework Microsoftu do:

- budowy agentów AI,
- orchestration,
- memory,
- pluginów,
- workflow AI.

11. Ollama

Ollama umożliwia:

uruchamianie LLM lokalnie

np:

- Llama,
- Mistral,
- DeepSeek,
- Gemma.

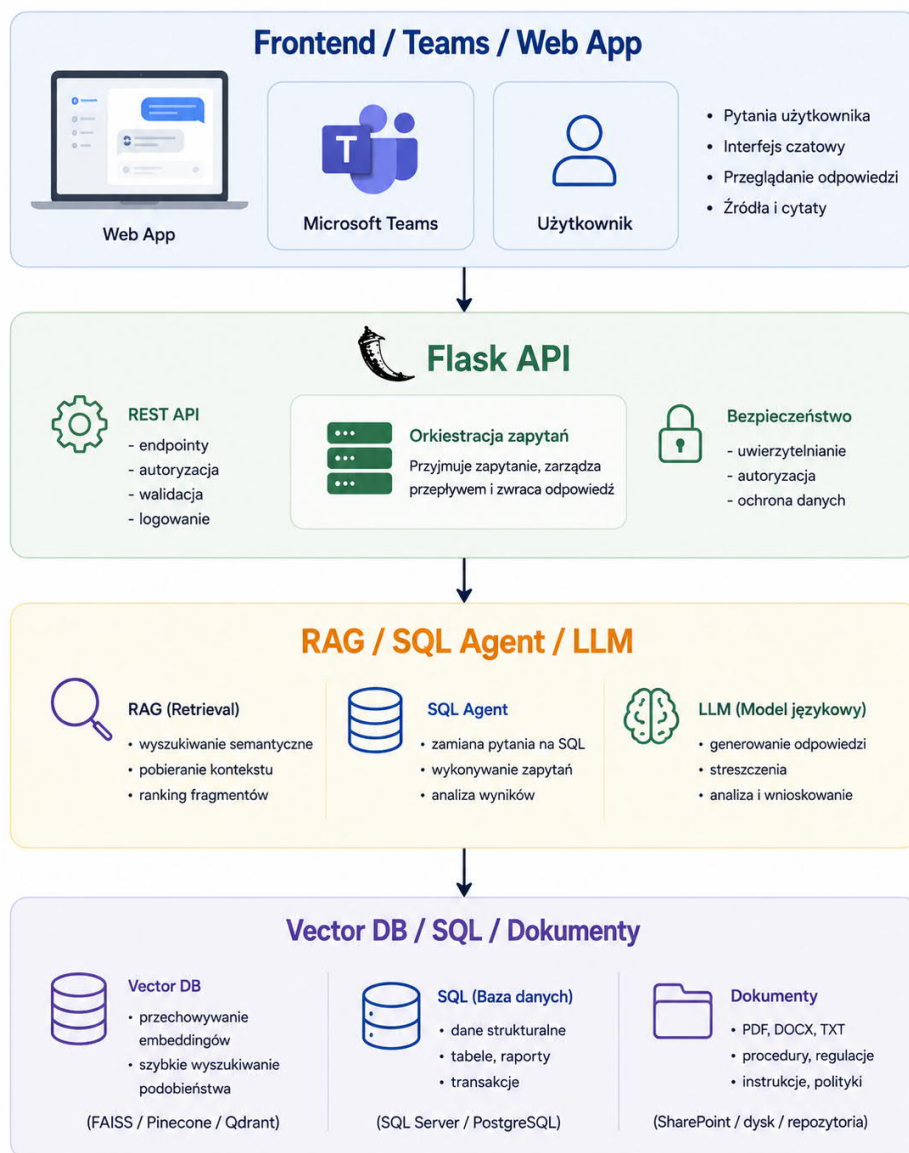
12. Flask API

warstwa komunikacyjna systemu AI czyli sposób, w jaki:

- frontend,
- aplikacje,
- systemy firmy,
- użytkownicy,
- inne serwisy

komunikują się z modelem AI.

Flask API udostępnia system AI jako:



Flask w systemach RAG

W architekturze RAG Flask zwykle:

- przyjmuje pytanie,
- wyszukuje embeddingi,
- pobiera dokumenty,
- buduje context,
- uruchamia LLM,
- zwraca odpowiedź.

Dlaczego API jest ważne?

Bo umożliwia integrację:

- z frontendem,
- CRM,
- ERP,
- Teams,
- Slack,
- aplikacją mobilną.

13. Integracja RAG + SQL + API

To najważniejszy element całego materiału.

Powstaje system:

- rozumiejący dokumenty,
- analizujący dane,
- komunikujący się z systemami,
- podejmujący akcje.

14. Dlaczego firmy wdrażają takie systemy?

Bo:

- automatyzują wiedzę,
- skracają czas analizy,
- przyspieszają decyzje,
- integrują dane z wielu systemów,
- umożliwiają self-service analytics,
- zmniejszają koszty operacyjne.

Najważniejsza idea całego materiału

Nowoczesny system AI to nie tylko:

LLM + prompt

ale:

- + retrieval,
- + embedding,
- + vector search,
- + orchestration,
- + integracja systemów,
- + pipeline produkcyjny.

Problem

Rozwiązanie: Hybrid Retrieval AI System SQL + RAG + LLM dla analizy danych zakupowych

System łączy dwa źródła wiedzy:

```
SQL → dane zakupowe: klienci, produkty, transakcje  
RAG → dokumenty: zasady rabatów, program premium, reklamacje  
LLM → łączy wyniki i generuje odpowiedź
```

Przykładowy scenariusz

Pytanie użytkownika:

```
Którzy klienci kwalifikują się do programu premium?
```

System działa tak:

1. RAG wyszukuje definicję programu premium.
2. Z dokumentu wynika: klient premium = wydatki > 5000 PLN.
3. SQL Agent liczy wydatki klientów.
4. LLM łączy definicję z wynikiem SQL.
5. Użytkownik dostaje odpowiedź w języku naturalnym.

Struktura danych SQL

```
customers(customer_id, name, city)
```

```
products(product_id, product_name, category, price)
```

```
purchases(purchase_id, customer_id, product_id, quantity,  
purchase_date)
```

Dokumenty dla RAG

```
premium_program.txt  
discount_rules.txt  
returns_policy.txt  
complaints.txt
```

Przykład premium_program.txt:

```
Program premium obejmuje klientów, którzy wydali więcej niż
5000 PLN w ciągu ostatnich 12 miesięcy. Klienci premium
otrzymują dodatkowe rabaty, priorytetową obsługę oraz darmową
dostawę.
```

Przykładowa odpowiedź systemu

Zgodnie z zasadami programu premium klient kwalifikuje się, jeśli wydał ponad 5000 PLN w ciągu ostatnich 12 miesięcy.

Na podstawie danych zakupowych warunków spełniają:

- Jan Kowalski – 7200 PLN
- Anna Nowak – 6100 PLN

Rola komponentów

SQL Agent

Liczy dane, wykonuje agregacje, filtruje rekordy.

RAG

Dostarcza reguły biznesowe i kontekst z dokumentów.

LLM

Łączy wynik SQL z kontekstem RAG i tworzy odpowiedź dla użytkownika.

Efekt końcowy

Budujemy prostego **AI Copilota zakupowego**, który odpowiada nie tylko na pytania liczbowe, ale także interpretuje dane według zasad zapisanych w dokumentach.

Kod do przykładu znajdują się w załączniku.

Zadanie do wykonania

Ćwiczenie projektowe AI Incident Management Copilot Hybrid Retrieval: SQL + RAG + LLM dla analizy incydentów IT

Temat

Projekt i implementacja systemu AI wspierającego analizę incydentów IT z wykorzystaniem Hybrid Retrieval, RAG, SQL Agent oraz modeli LLM

Cel ćwiczenia

Celem projektu jest stworzenie systemu AI, który:

- analizuje incydenty IT zapisane w bazie danych,
- korzysta z dokumentacji technicznej i procedur bezpieczeństwa,
- odpowiada na pytania użytkownika w języku naturalnym,
- wspiera dział IT oraz SOC (Security Operations Center).

Problem biznesowy

Firma posiada dużą liczbę incydentów IT oraz dokumentów bezpieczeństwa. Administratorzy i analitycy SOC potrzebują systemu, który umożliwi szybkie wyszukiwanie informacji, analizę incydentów oraz generowanie odpowiedzi w języku naturalnym.

Dane strukturalne (SQL)

- zgłoszenia incydentów,
- logi błędów,
- serwery,
- użytkowników,
- statusy zgłoszeń.

Dane niestrukturalne (RAG)

- procedury bezpieczeństwa,
- instrukcje recovery,
- dokumentację systemów,
- polityki bezpieczeństwa,
- playbooki SOC.

System ma odpowiadać na pytania:

1. Ile incydentów krytycznych wystąpiło w maju?
2. **Jak** wygląda procedura obsługi ransomware?
3. Które serwery miały najwięcej awarii i **jaka jest** rekomendowana procedura recovery?
4. Czy incydent spełnia definicję krytycznego incydentu?

Hybrid Retrieval

SQL

Pobiera:

- incydenty,
- logi,
- agregacje,
- statystyki.

RAG

Pobiera:

- procedury,
- definicje,
- instrukcje recovery,
- polityki bezpieczeństwa.

LLM

Łączy:

- dane SQL,
- wiedzę z dokumentów,
- generuje odpowiedź.

Przykład działania

Pytanie użytkownika

Które serwery miały najwięcej awarii i jaka jest procedura recovery?

SQL Agent

Generuje:

```
SELECT server_name, COUNT(*) AS failures
FROM incidents
GROUP BY server_name
ORDER BY failures DESC
LIMIT 3;
```

RAG

Znajduje dokument:

Recovery procedure:

1. Restart service
2. Verify logs
3. Restore backup
4. Escalate to SOC

LLM

Generuje odpowiedź:

Najwięcej awarii wystąpiło na serwerach:
srv-db-01 oraz srv-auth-02.

Zgodnie z procedurą recovery należy:

1. Zweryfikować logi,

2. Odtworzyć backup,
3. Eskalować incydent do SOC.

Struktura bazy danych

Tabela incidents

```
CREATE TABLE incidents (  
    incident_id INTEGER PRIMARY KEY,  
    server_name TEXT,  
    severity TEXT,  
    status TEXT,  
    created_at TEXT  
);
```

Tabela users

```
CREATE TABLE users (  
    user_id INTEGER PRIMARY KEY,  
    username TEXT,  
    department TEXT  
);
```

Tabela systems

```
CREATE TABLE systems (  
    system_id INTEGER PRIMARY KEY,  
    system_name TEXT,  
    owner TEXT  
);
```

Dokumenty dla RAG

Student powinien przygotować:

incident_response.txt

recovery_procedures.txt

security_policy.txt
ransomware_playbook.txt
critical_incident_definition.txt

Przykład dokumentu

critical_incident_definition.txt

Krytyczny incydent bezpieczeństwa
to incydent powodujący:

- niedostępność usług,
- utratę danych,
- naruszenie bezpieczeństwa,
- ransomware,
- wyciek danych.

Zakres implementacji

1. SQL Agent

Student implementuje:

- generowanie SQL,
- wykonywanie zapytań,
- agregacje,
- analizy incydentów.

2. RAG Pipeline

Student implementuje:

- chunking,
- embedding,
- FAISS,
- semantic search,
- retrieval.

3. Hybrid Retrieval

System:

- pobiera dane z SQL,
- pobiera wiedzę z dokumentów,
- przekazuje wszystko do LLM.

4. Flask API

Endpointy:

POST /ask

POST /sql

POST /rag

GET /health

5. LLM Layer

LLM:

- interpretuje incydenty,
- generuje odpowiedzi,
- tworzy rekomendacje.

Technologie

- Python,
- Flask,
- SQLite,
- SentenceTransformers,
- FAISS,
- Ollama / Llama / Mistral,
- Pandas,
- SQLAlchemy

Przykładowe pytania

SQL only

Ile incydentów wystąpiło w czerwcu?

RAG only

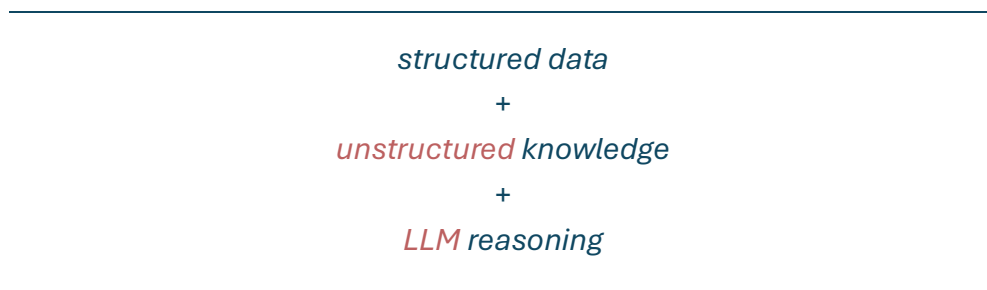
Jak wygląda procedura recovery?

Hybrid

Które serwery miały najwięcej incydentów
i jaka procedura obowiązuje dla incydentów krytycznych?

Co student powinien zrozumieć?

Projekt pokazuje:



czyli nowoczesną architekturę:

Hybrid Retrieval AI System

Efekt końcowy

Budujemy prototyp:

AI SOC Copilot czyli system przypominający rozwiązania:

- SIEM AI,
- Security Copilot,
- enterprise incident assistants,
- AI SOC analyst tools.

Przedstaw w postaci raportu pełną architekturę enterprise AI systemu oraz:

- Odpowiedzi systemu,
- Retrieved documents (dokumenty odnalezione przez RAG),
- SQL query,
- Generated response,
- Kod rozwiązania.